

Universidad del Valle de Guatemala

Facultad de Ingeniería

Computación Paralela y Distribuida



Explicación PCAM

Laboratorio PCAM + OpenMP

Simulación gravitacional N-Body en 2D

Dilary Sarahí Cruz López - 231010

Gerardo Fernandez - 23763

Jose Ruiz - 23719

Guatemala, 30 de septiembre de 2026

Repositorio: github.com/jruiz002/lab-pcam-openmp-nbody

1. Resumen y entorno de medición

Se implementaron tres versiones en C (C17) con OpenMP de una simulación gravitacional N-Body en dos dimensiones y se analizaron con la metodología PCAM:

- **A** — `nbody_seq.c`: versión secuencial (línea base).
- **B** — `nbody_parallel.c`: paralela directa; cada cuerpo acumula su propia fuerza.
- **C** — `nbody_optimized.c`: cada par (i, j) se calcula una sola vez usando $F_{ij} = -F_{ji}$.

Todas las versiones usan `seed = 42`, $N = 5000$ cuerpos, 10 pasos y $\Delta t = 0.01$, y miden el tiempo con `omp_get_wtime()`. Cada configuración se ejecutó tres veces y se reporta el promedio (`results.csv`, generado por `run_experiments.sh`). El código fuente completo está en el repositorio github.com/jruiz002/lab-pcam-openmp-nbody.

Tabla 1: Entorno de medición.

Procesador	Apple M2 Pro, 10 núcleos (6 de rendimiento + 4 de eficiencia)
Memoria	16 GB
OpenMP	<code>omp_get_num_procs() = 10</code>
Compilador	<code>GCC 14.2, -O2 -std=c17 -fopenmp</code>

Los 8 hilos se reparten entre núcleos heterogéneos y una corrida completa dura menos de 0.4s, por lo que la variación entre corridas es de varios por ciento; diferencias menores a $\sim 5\%$ no se consideran concluyentes.

La mejor configuración obtenida fue la versión C con `reduction, schedule(dynamic, 64)` y 8 hilos: 0.069s frente a 0.355s de la secuencial, es decir, $S = 5.17$ y $E = 0.65$, con el mismo checksum que la versión secuencial.

2. Diseño PCAM

Partition

La unidad paralelizable es el *cálculo de la fuerza total sobre el cuerpo i* en un paso, es decir, una iteración del ciclo externo. La unidad mínima de trabajo es una interacción (i, j) , pero no se usa como tarea porque su costo (~ 20 flops) es mucho menor que el de despacharla. Las tareas de un mismo paso son independientes entre sí, ya que todas leen las posiciones del instante t . Entre pasos existe una dependencia estricta ($t + 1$ necesita el estado final de t), y dentro de un paso hay una barrera entre calcular fuerzas y mover cuerpos. En B y C, el final del `parallel for` proporciona esa barrera de forma implícita.

Communication

Los datos `x`, `y` y `mass` de todos los cuerpos son de *solo lectura* durante el cálculo de fuerzas y se comparten sin protección. En B cada hilo escribe únicamente `vx[i]` y `vy[i]` de los cuerpos que le fueron asignados, por lo que no existen condiciones de carrera. En C, en cambio, la tercera ley de Newton hace que el par (i, j) escriba en los acumuladores de *dos* cuerpos (`fx[i]` y `fx[j]`), y j puede estar siendo procesado por otro hilo: ahí se produce la carrera. Los acumuladores `fx` y `fy` son lo único que debe sincronizarse.

Agglomeration

Se aglomeran todas las interacciones de un cuerpo en una sola tarea (unos N pares por tarea) y, mediante el parámetro `chunk`, varios cuerpos consecutivos por asignación del planificador. Se compararon chunks de 1, 8, 64 y 512, además del valor por defecto, para observar el efecto de la granularidad.

Mapping

La asignación se realiza con `schedule(runtime)`, lo que permite comparar `static`, `dynamic` y `guided` sin recompilar. En B todas las tareas cuestan lo mismo ($N - 1$ interacciones), así que `static` sería la opción natural. En C el ciclo $j = i+1..N$ es *triangular*: la tarea i cuesta $N - i - 1$ pares y, con `static` por bloques y 8 hilos, el primer hilo recibe aproximadamente 15 veces más trabajo que el último. Este desbalance explica por qué `dynamic` con chunks pequeños resulta mucho mejor en C.

3. Estrategia OpenMP y condiciones de carrera

Versión B. Se paraleliza el ciclo externo con `schedule(runtime)`. Las variables del ciclo interno se declaran dentro del ciclo, por lo que son privadas de cada hilo, y cada hilo escribe solo la velocidad de sus propios cuerpos. El movimiento de posiciones se hace en un segundo `parallel for`, después de la barrera implícita:

```
#pragma omp parallel for schedule(runtime)
for (int i = 0; i < N; i++) {
    double fx = 0.0, fy = 0.0; /* privadas del hilo */
    for (int j = 0; j < N; j++) { /* ... solo lee bodies[j] ... */ }
    bodies[i].vx += (fx / bodies[i].mass) * dt; /* solo escribe su i */
    bodies[i].vy += (fy / bodies[i].mass) * dt;
} /* barrera implícita */
#pragma omp parallel for schedule(runtime)
for (int i = 0; i < N; i++) { bodies[i].x += bodies[i].vx * dt; /* ... */ }
```

Versión C. Se probaron tres formas de resolver la carrera sobre `fx[i]`, `fy[i]`, `fx[j]` y `fy[j]`, compiladas a partir del mismo código fuente (`-DSYNC_ATOMIC`, `-DSYNC_CRITICAL` o, por defecto, `reduction`). La tabla 2 resume los resultados.

Tabla 2: Estrategias de sincronización en la versión C (8 hilos, `dynamic`, `chunk 64`).

Estrategia	Tiempo (s)	Speedup
<code>reduction(+:fx[:N], fy[:N])</code>	0.0687	5.17
<code>atomic (4 por par)</code>	0.5463	0.65
<code>critical (1 por par)</code>	2.5890	0.14

Con `atomic` se realizan cerca de 50 millones de operaciones atómicas por paso (12.5×10^6 pares $\times$ 4), y el programa resulta más lento que el secuencial; `critical` serializa por completo el núcleo del cálculo. Se **eligió `reduction`** porque elimina la contención: cada hilo acumula en una copia privada de `fx` y `fy`, que se suman al final. Su costo es memoria adicional ($2N$ `double` por hilo, unos 80 KB)

y una fase de combinación $O(N \cdot p)$, ambos despreciables frente al trabajo $O(N^2)$. La diferencia numérica máxima frente a la versión secuencial es 2.2×10^{-8} .

4. Preguntas de análisis

1. **Unidad mínima.** Una interacción (i, j) , de unos 20 flops. Como tarea individual, el costo de despacharla y sincronizar supera con creces el del cálculo; por eso se aglomeran las N interacciones de un cuerpo en una tarea.
2. **Datos compartidos.** Son de solo lectura `x`, `y` y `mass` durante el cálculo de fuerzas. Requieren protección, en C, los acumuladores `fx` y `fy` (resuelto con `reduction`). En B no hay escrituras compartidas.
3. **Efecto del chunk.** Un chunk muy grande perjudica: con 8 hilos y `static` 512, B pasa de 0.111 a 0.201 s (con `dynamic` 512: 0.117 s en B y 0.096 s en C), porque quedan unos 10 bloques y el último desbalancea la carga. Un chunk de 1 en `dynamic` paga más overhead de planificación (chunk 1 frente a 8: 0.129 vs. 0.093 s en B y 0.073 vs. 0.069 s en C). El óptimo es intermedio (8–64), donde el overhead se amortiza sin perder balance.
4. **static vs. dynamic vs. guided.** En ambas versiones `dynamic` fue el mejor (B 0.089 s y C 0.069 s con chunk 64, frente a 0.111 y 0.116 s con `static`). En B, con carga uniforme, la ventaja ($\sim 20\%$) se atribuye a que `dynamic` absorbe la interferencia del sistema operativo y la heterogeneidad de los núcleos. En C la carga triangular amplifica el efecto: `static` por defecto deja al primer hilo con unas 15 veces la carga del último. `guided` fue intermedio en B (0.099 s) y peor en C con chunk por defecto (0.123 s), porque sus primeros bloques son grandes y caen en las filas más costosas.
5. **Condición de carrera en C.** El par (i, j) actualiza `fx[i]`, `fy[i]`, `fx[j]` y `fy[j]`; otro hilo que procesa j como su propio i escribe el mismo `fx[j]` en paralelo, y se pierden actualizaciones. Se resolvió con acumuladores privados por hilo (`reduction` sobre arreglos) y se comparó con `atomic` y `critical`.
6. **¿Menos operaciones implica más rapidez?** No siempre. C hace la mitad de interacciones y, con `dynamic` 64, supera a B (0.069 vs. 0.089 s). Sin embargo, con `static` por defecto y 2 hilos es más lenta (0.346 vs. 0.281 s) por el desbalance triangular y el costo de inicializar y combinar las copias privadas; con `guided` por defecto también pierde (0.123 vs. 0.099 s). Con `atomic` (0.546 s) o `critical` (2.59 s) es incluso más lenta que la secuencial: domina la sincronización.
7. **Speedup y eficiencia.** Con 2 hilos: $S = 1.26$, $E = 0.63$ (B) y $S = 1.54$, $E = 0.77$ (C). Con 4 hilos: $S = 2.36$, $E = 0.59$ (B) y $S = 3.03$, $E = 0.76$ (C). Con 8 hilos: $S = 3.20$, $E = 0.40$ (B, `static`); $S = 4.01$, $E = 0.50$ (B, `dynamic` 64) y $S = 5.17$, $E = 0.65$ (C). El escalamiento deja de ser lineal desde 2 hilos y se degrada más con 8, sobre todo porque el M2 Pro solo tiene 6 núcleos de alto rendimiento. También pesan el costo fijo de crear el equipo de hilos y la parte secuencial (ley de Amdahl), dado que el tiempo total es de unos 0.1 s.
8. **¿64 núcleos implican 8 veces más rapidez?** No. Con 8 hilos la eficiencia ya bajó a 0.40–0.65 y el speedup fue de 3.2 a 5.2, no de 8. Con 64 hilos y $N = 5000$ (78 cuerpos por hilo), el overhead de planificación, la creación de hilos y, en C, la combinación de 64 copias de $2N$ valores crecerían mientras el trabajo por hilo disminuye. Para escalar de forma útil habría que aumentar N , ya que el trabajo crece como $O(N^2)$.

5. Conclusiones

- El paralelismo por cuerpo es natural para N-Body: las tareas son independientes dentro de un paso y la única sincronización necesaria es la barrera entre fases y entre pasos.
- La mejor configuración fue la versión C con **reduction**, **dynamic**, chunk 64 y 8 hilos: 0.069s, speedup de 5.17 y eficiencia de 0.65, con resultados numéricamente equivalentes a la versión secuencial.
- La estrategia contra la condición de carrera fue el factor más determinante: **reduction** ($5.18\times$) frente a **atomic** ($0.65\times$) y **critical** ($0.14\times$).
- El schedule y el chunk importan: **dynamic** con chunks de 8 a 64 fue el mejor en ambas versiones, mientras que chunks de 512 y **guided** por defecto fueron los peores, especialmente con la carga triangular de C.
- Limitaciones: con $N = 5000$ y 10 pasos las corridas duran menos de 0.4s, por lo que existe ruido de medición, y los núcleos heterogéneos del M2 Pro limitan la eficiencia con 8 hilos.